

I. QUESTION (ALGORITHM ANALYSIS)

Given the following function

```

1  int algorithm1(int N, int k){
2      if (N == 0)
3          return 0;
4      int sum = 0;
5      for (int i = 0; i < N; i++)
6          sum++;
7      for (int i = 0; i < N; i++)
8          for (int j = 0; j < sqrt(N); j++)
9              sum++;
10     if (sum % 2 == 0)
11         sum += algorithm1(N / 2);
12     else
13         sum -= algorithm1(N / 2);
14     return sum;
15 }
```

What is the time complexity of algorithm1? Explain your result. Use the master theorem.

II. QUESTION (ALGORITHM ANALYSIS)

Given the following function

```

1  int algorithm2(int N){
2      int i, j, k, sum = 0;
3      for (i = 1; i <= N; i++)
4          sum++;
5      for (i = 1; i <= N; i++)
6          for (j = N; j >= i; j--)
7              sum++;
8      for (i = 1; i <= N; i++)
9          for (j = 1; j <= N; j++)
10             for (k = j; k <= N - j; k++)
11                 sum++;
12     return sum;
13 }
```

What does the function algorithm2 return in terms of N?

III. QUESTION (LINKEDLIST)

Write the method

```
DoublyLinkedList getEvenOnes()
```

for doubly linked lists, which returns a new doubly linked list with even indexed nodes (2, 4, ...) from the original list. Your linked list should contain new nodes, not the same nodes in the original linked list. The first node has index 1. You are not allowed to use any linked list methods, just attributes, constructors, getters and setters.

IV. QUESTION (LINKEDLIST)

Write the method

```
boolean containsOnlyDuplicates()
```

which returns true if the singly linked list only contains duplicates, that is, every datum (number) occurs only twice. Important warning, the duplicate elements may not be adjacent. You are not allowed to use any singly linked list methods, just attributes, constructors, getters and setters.

V. QUESTION (STACK)

Write the method

```
void removeOddIndexed()
```

which removes only the odd indexed (1, 3, ...) elements from the stack. The first element at the bottom has index 1. You are only allowed to use pop, push, isEmpty functions (Hint: Use external stack). You are not allowed to use any stack attributes such as *N*, *top*, *array* etc. You can solve the question for any stack implementation.

VI. QUESTION (STACK)

Write the method

```
Element pop(int k)
Node pop(int k)
```

which pops the *k*'th element from the top and returns it. pop(1) is equal to the original pop, that is, the first element at the top has index 1. You are not allowed to use any stack methods and any external stacks, just attributes, constructors, getters and setters. Write the code for both array and linked list implementations.